

TCX1001 Introduction to Computing

In-person Lecture 4

Interpreter



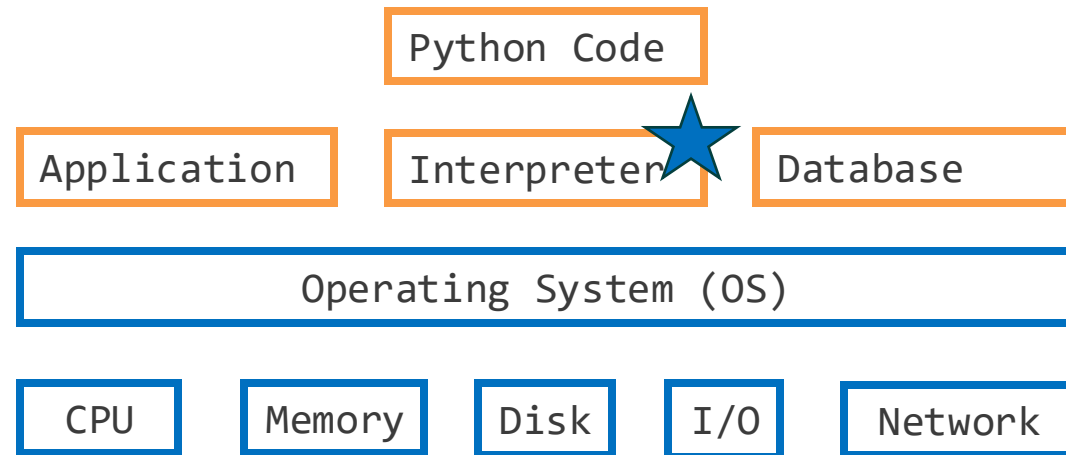
Dr JIANG Kan Michael



Mr KITAHARA Yutaro

Objective

- Parsing code
 - Reading Grammer
- Interpreting code
- Python VM



Grammar

- $\langle \text{expression} \rangle ::= \langle \text{term} \rangle \{ ('+' | '-') \langle \text{term} \rangle \}$
- $\langle \text{term} \rangle ::= \langle \text{factor} \rangle \{ ('*' | '/') \langle \text{factor} \rangle \}$
- $\langle \text{factor} \rangle ::= \langle \text{number} \rangle \mid '(' \langle \text{expression} \rangle ')'$

Is the expression grammatically correct?

expr = "3"

Grammar

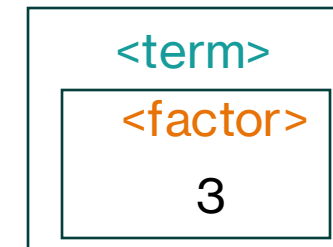
- $\langle \text{expression} \rangle ::= \langle \text{term} \rangle \{ ('+' | '-') \langle \text{term} \rangle \}$
- $\langle \text{term} \rangle ::= \langle \text{factor} \rangle \{ ('*' | '/') \langle \text{factor} \rangle \}$
- $\langle \text{factor} \rangle ::= \langle \text{number} \rangle \mid '(' \langle \text{expression} \rangle ')'$

Is the expression grammatically correct? Yes

- 3 is a valid term because:
- 3 is a valid factor because:
- 3 is a valid number.

expr = "3"

OK

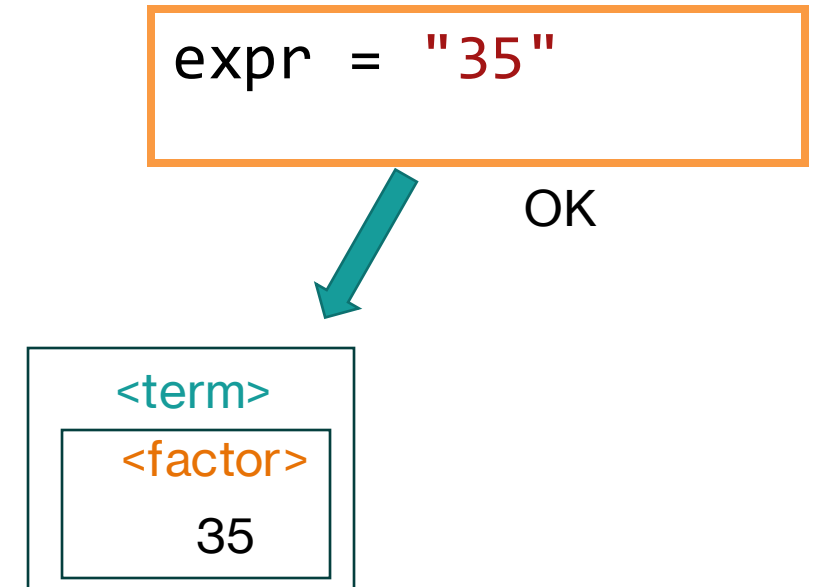


Grammar

- $\langle \text{expression} \rangle ::= \langle \text{term} \rangle \{ ('+' | '-') \langle \text{term} \rangle \}$
- $\langle \text{term} \rangle ::= \langle \text{factor} \rangle \{ ('*' | '/') \langle \text{factor} \rangle \}$
- $\langle \text{factor} \rangle ::= \langle \text{number} \rangle \mid '(' \langle \text{expression} \rangle ')'$

Is the expression grammatically correct? Yes

- 35 is a valid term because:
- 35 is a valid factor because:
- 35 is a valid number.



Grammar

- $\langle \text{expression} \rangle ::= \langle \text{term} \rangle \{ ('+' | '-') \langle \text{term} \rangle \}$
- $\langle \text{term} \rangle ::= \langle \text{factor} \rangle \{ ('*' | '/') \langle \text{factor} \rangle \}$
- $\langle \text{factor} \rangle ::= \langle \text{number} \rangle \mid '(' \langle \text{expression} \rangle ')'$

Is the expression grammatically correct?

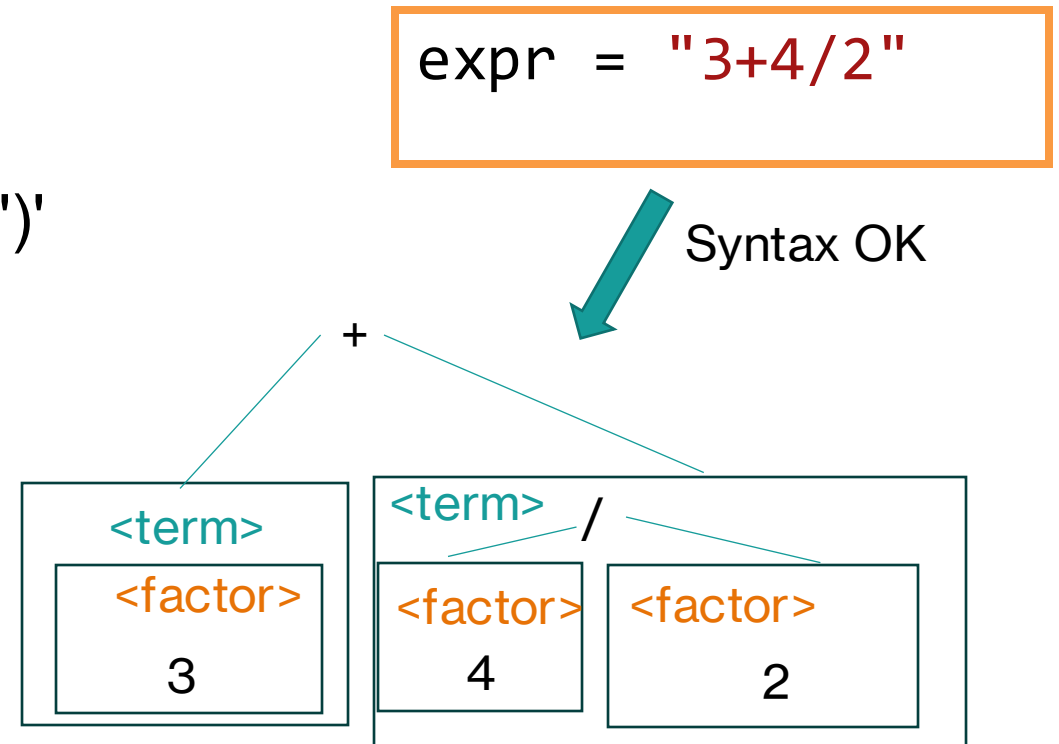
expr = "3+4/2"

Grammar

- $\langle \text{expression} \rangle ::= \langle \text{term} \rangle \{ ('+' | '-') \langle \text{term} \rangle \}$
- $\langle \text{term} \rangle ::= \langle \text{factor} \rangle \{ ('*' | '/') \langle \text{factor} \rangle \}$
- $\langle \text{factor} \rangle ::= \langle \text{number} \rangle \mid '(' \langle \text{expression} \rangle ')'$

Is the expression grammatically correct? Yes

- 3 is a valid term, and 4/2 is a valid 2nd term:
- 4 is a valid factor and 2 is a valid 2nd factor:
- 4 is a valid number and 2 is a valid number.

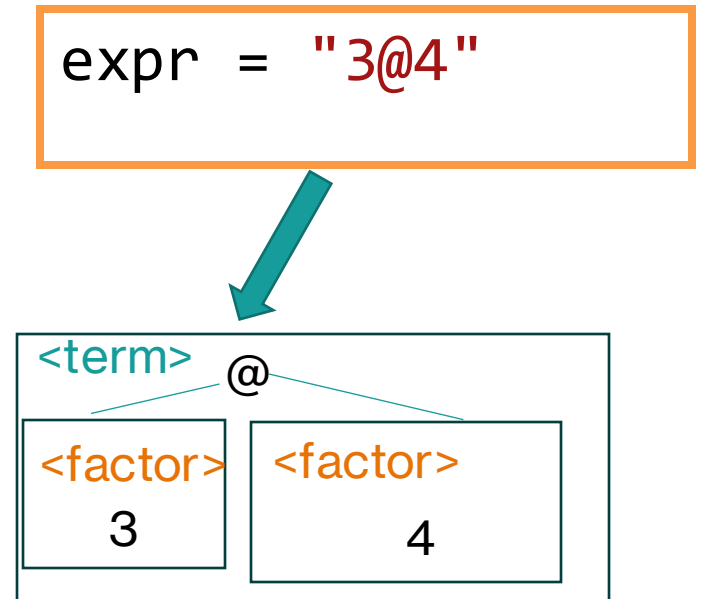


Grammar

- $\langle \text{expression} \rangle ::= \langle \text{term} \rangle \{ ('+' | '-') \langle \text{term} \rangle \}$
- $\langle \text{term} \rangle ::= \langle \text{factor} \rangle \{ '@' \langle \text{factor} \rangle \}$
- $\langle \text{factor} \rangle ::= \langle \text{number} \rangle \mid '(' \langle \text{expression} \rangle ')'$

Is the expression grammatically correct? Yes

- 3@4 is a valid term:
- 3 is a valid factor and 4 is a valid 2nd factor:
- 3 is a valid number and 4 is a valid number.



Grammar

- $\langle \text{expression} \rangle ::= \langle \text{term} \rangle \{ ('+' | '-') \langle \text{term} \rangle \}$
- $\langle \text{term} \rangle ::= \langle \text{factor} \rangle \{ '@' \langle \text{factor} \rangle \}$
- $\langle \text{factor} \rangle ::= \langle \text{number} \rangle \mid '(' \langle \text{expression} \rangle ')'$

Is the expression grammatically correct?

expr = "3 - (2@4)"

Grammar

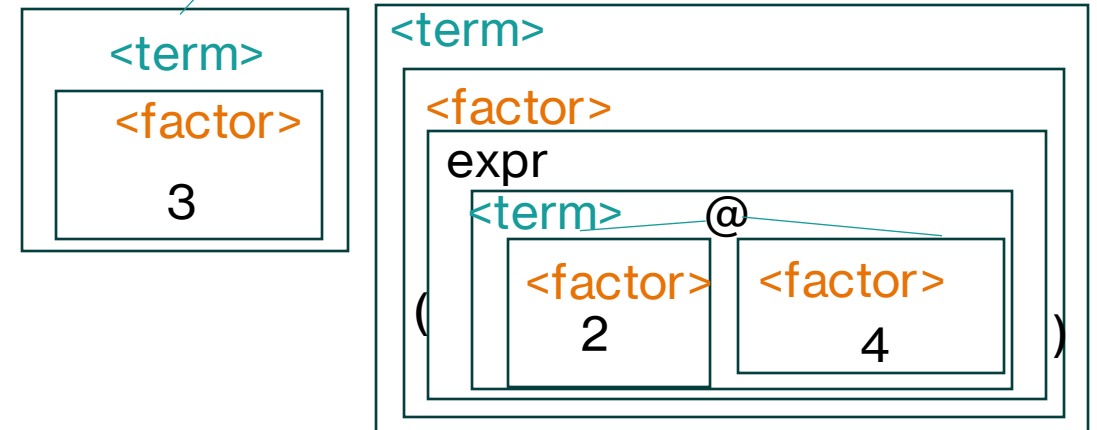
- $\langle \text{expression} \rangle ::= \langle \text{term} \rangle \{ ('+' | '-') \langle \text{term} \rangle \}$
- $\langle \text{term} \rangle ::= \langle \text{factor} \rangle \{ '@' \langle \text{factor} \rangle \}$
- $\langle \text{factor} \rangle ::= \langle \text{number} \rangle \mid '(' \langle \text{expression} \rangle ')'$

Is the expression grammatically correct? Yes

- 3 is a valid term and (2@4) is a valid 2nd term:
- (2@4) is a valid factor:
- 2@4 is a valid expression
- 2@4 is a valid term
- 2 is a valid factor and 4 is a valid factor

expr = "3 - (2@4)"

Syntax OK



Grammar

- $\langle \text{expression} \rangle ::= \langle \text{term} \rangle \{ ('+' | '-') \langle \text{term} \rangle \}$
- $\langle \text{term} \rangle ::= \langle \text{factor} \rangle \{ '@' \langle \text{factor} \rangle \}$
- $\langle \text{factor} \rangle ::= \langle \text{number} \rangle \mid '(' \langle \text{expression} \rangle ')'$

Is the expression grammatically correct?

expr = "2@(4@)"

Grammar

- $\langle \text{expression} \rangle ::= \langle \text{term} \rangle \{ ('+' | '-') \langle \text{term} \rangle \}$
- $\langle \text{term} \rangle ::= \langle \text{factor} \rangle \{ '@' \langle \text{factor} \rangle \}$
- $\langle \text{factor} \rangle ::= \langle \text{number} \rangle \mid '(' \langle \text{expression} \rangle ')'$

Is the expression grammatically correct? No!

- Is $(4@)$ a valid factor?
- Is $4@$ a valid expression? No!

How to fix it?

$2@(4@1)$

expr = "2@(4@)"

String Pattern

- If we simply convert expr to list of characters:
 - `list(expr)`
 - `['3', '*', '(', '1', '1', '-', '5', ')', '/', '2']`
 - But, we want "11" as one number!
- Instead, space out each operator:
 - `"3 * ( 11 - 5 ) / 2"`
- What is the source list?
- Control variable?
- Accumulator?

```
expr = "3*(11-5)/2"
```

```
def parse(expr):  
    operators = list("+-*/()")  
    pass
```

```
print(parse(expr))  
# ['3', '*', '(', '11', '-', '5', ')', '/', '2']
```

String Pattern

- What is the source list?
 - The expression, eg "3*(11-5)/2"
- Control variable?
 - Variable ch represents each character
 - If ch is an op, add spaces before and after
 - If ch is not an op, append ch to accumulator
- Accumulator variable: res
- Finally, simply split(), by spaces

```
expr = "3*(11-5)/2"

def parse(expr):
    operators = list("+-*/()")
    res = ""
    for ch in expr:
        if ch in operators:
            res += (" " + ch + " ")
        else:
            res += ch
    return res.split()

print(parse(expr))
# ['3', '*', '(', '11', '-', '5', ')', '/', '2']
```

[https://pythontutor.com/render.html#code=expr%20%3D%20'223'*%2811-5%29%2F%22%20Adef%20parse%28expr%29%3A%0A%20%20operators%20%3D%20list%28%22%2B-*%28%29%22%29%6A%20%20res%20%3D%20%22%22%20A%20%20%20for%20ch%20in%20expr%3A%0A%20%20%20%20if%20ch%20in%20operators%3A%0A%20%20%20%20%20%20%20res%20%2B%3D%20%28%22%20%22%20%20%20ch%20%2B%20%22%20%22%29%0A%20%20%20%20else%3A%0A%20%20%20%20%20%20%20res%20%2B%3D%20ch%0A%20%20return%20res.split%28%29%0Aprint%28parse%28expr%29%29%0A%23%20%5B'3'%20**'%20%28'%20'11'%20'-'%20'5'%20%29'%20'\)'%20%20%5D&cumulative=false&curlInstr=39&heapPrimitives=nevernest&mode=display&origin=opt-front&isQv=311&rawInputLstJSON=%5B%5D&textReferences=false](https://pythontutor.com/render.html#code=expr%20%3D%20'223'*%2811-5%29%2F%22%20Adef%20parse%28expr%29%3A%0A%20%20operators%20%3D%20list%28%22%2B-*%28%29%22%29%6A%20%20res%20%3D%20%22%22%20A%20%20%20for%20ch%20in%20expr%3A%0A%20%20%20%20if%20ch%20in%20operators%3A%0A%20%20%20%20%20%20%20res%20%2B%3D%20%28%22%20%22%20%20%20ch%20%2B%20%22%20%22%29%0A%20%20%20%20else%3A%0A%20%20%20%20%20%20%20res%20%2B%3D%20ch%0A%20%20return%20res.split%28%29%0Aprint%28parse%28expr%29%29%0A%23%20%5B'3'%20**'%20%28'%20'11'%20'-'%20'5'%20%29'%20')'%20%20%5D&cumulative=false&curlInstr=39&heapPrimitives=nevernest&mode=display&origin=opt-front&isQv=311&rawInputLstJSON=%5B%5D&textReferences=false)

String Pattern, Alternative Solution

- Control variable:
 - Variable `ch` represents each character
 - Instead of check if `ch` is an op, **is `ch` a digit?**
- Need an additional variable to track the current number
- Try it out

```
expr = "3*(11-5)/2"

def parse(expr):
    num = ""
    res = ""
    for ch in expr:
        if ch.isdigit():
            ???

    return res.split()
```

String Pattern, Alternative Solution

- Control variable: ch
- Accumulator: res
- Additional variable num tracks current num.
 - If ch is digit, grow the current num
 - Else:
 - Append current non-empty num to accumulator
 - Append op to accumulator
- Post-process:
 - Append current non-empty num to accumulator

<https://pythontutor.com/render.html#code=expr%20%3D%20%223%2A%2811-5%29%2F2%22%20Adef%20parse%28expr%29%3A%0A%20%20num%20%3D%20%22%22%20A%20%20res%20%3D%20%22%22%20A%20%20for%20ch%20in%20expr%3A%0A%20%20%20if%20ch.isdigit%28%29%3A%0A%20%20%20%20%20%20num%20%2B%3D%20ch%0A%20%20%20%20else%3A%0A%20%20%20%20%20%20if%20num%3A%0A%20%20%20%20%20%20%20res%20%2B%3D%20num%20%2B%20%22%20%22%29%0A%20%20%20%20%20%20%20num%20%20%20%20%20%20%20res%20%2B%3D%20%20%20res%20%2B%3D%20%28ch%20%2B%20%22%20%22%29%0A%20%20if%20num%3A%0A%20%20%20%20%20res%20%2B%3D%20num%20%2B%20%22%20%22%29%0A%20%20return%20res.split%28%29%0Aprint%28parse%28expr%29%29%0A&cumulative=false&curlInstr=52&heapPrimitives=nevernest&mode=display&origin=opt-frontent.js&py=311&rawInputLstJSON=%5B%5D&textReferences=false>

```
expr = "3*(11-5)/2"

def parse(expr):
    num = ""
    res = ""
    for ch in expr:
        if ch.isdigit():
            num += ch
        else:
            if num:
                res += (num + " ")
                num = ""
            res += (ch + " ")
    if num:
        res += (num + " ")
    return res.split()
```


Expression with decimal point

- Use the least amount of change to process **decimal point, part of a number.**
- Try it out

```
expr = "3*(11.0-5)/2"

def parse(expr):
    num = ""
    res = ""
    for ch in expr:
        if ch.isdigit():
            num += ch
        else:
            if num:
                res += (num + " ")
                num = ""
            res += (ch + " ")
    if num:
        res += (num + " ")
    return res.split()
```

Expression with decimal point

- Use the least amount of change to process **decimal point, part of a number.**

```
expr = "3*(11.0-5)/2"

def parse(expr):
    num = ""
    res = ""
    for ch in expr:
        if ch.isdigit() or ch=='.':
            num += ch
        else:
            if num:
                res += (num + " ")
                num = ""
            res += (ch + " ")
    if num:
        res += (num + " ")
    return res.split()
```

Parse Expression with variable names

- Assume variable name is alphabet only
- How do you modify the existing parse()?

```
expr = "3*(abc-5)/b"

def parse(expr):
    var_name = ""
    num = ""
    res = ""
    for ch in expr:
        if ch.isdigit() or ch=="." :
            num += ch
        elif ch.isalpha():
            ???
        else:
            if num:
                res += (num + " ")
                num = ""
            if var_name:
                ???
            res += (ch + " ")
    if num:
        res += (num + " ")
    if var_name:
        ???
    return res.split()

print(parse(expr))
# ['3', '*', '(', 'abc', '-', '5', ')', '/', 'b']
```

Parsing variable names

- Additional variable `var_name` tracks current variable.
 - If `ch` is alpha, grow the current variable
 - Else:
 - Append current non-empty `var_name` to accumulator
- Post-process:
 - Append current non-empty `var_name` to accumulator

```
expr = "3*(abc-5)/b"
def parse(expr):
    var_name = ""
    num = ""
    res = ""
    for ch in expr:
        if ch.isdigit() or ch=="." :
            num += ch
        elif ch.isalpha():
            var_name += ch
        else:
            if num:
                res += (num + " ")
                num = ""
            if var_name:
                res += (var_name + " ")
                var_name = ""
            res += (ch + " ")
    if num:
        res += (num + " ")
    if var_name:
        res += (var_name + " ")
    return res.split()
```

Evaluate Simple Expression

- Input is a list of tokens, always processed from left to right.
- Return the calculated result
- Do we know how many iteration in advance?
Use for loop? Or while loop?
- In each iteration, process the first 3 tokens.
- Try it out.

```
# input: ['11.4', '-', '5', '+', '2']  
def eval_simple_expr(tokens):  
    pass  
  
# expected: 8.4
```

Evaluate Simple Expression

- In each iteration, process the first 3 tokens.
 - These are num1, op, num2
 - The intermediate result should be inserted back to the list of tokens

[illegible]

```
# input: ['11.4', '-', '5', '+', '2']
def eval_simple_expr(tokens):
    while len(tokens) >= 3:
        num1 = float(tokens.pop(0))
        op = tokens.pop(0)
        num2 = float(tokens.pop(0))
        if op == "+":
            tokens.insert(0, str(num1 + num2))
        elif op == "-":
            tokens.insert(0, str(num1 - num2))
        elif op == "*":
            tokens.insert(0, str(num1 * num2))
        elif op == "/":
            tokens.insert(0, str(num1 / num2))
    return float(tokens[0])
# expected: 8.4
```

Evaluate Expression

- Given a well-formed expression:
 - Parenthesis are always properly closed
 - Can call `eval_simple_expr()`
- Source list? for loop? while loop?
- Control variable?
- Accumulator?
- If it is not a ')', what should we do?
- If it is a ')', what should we do?

```
def eval_expr(tokens):  
    pass  
  
print(eval_expr(parse("2*(3.4-(1.2+0.2))/0.5")))  
# 8.0
```

Evaluate Expression

- Use for loop, loop through each token (e)
- Accumulator variable stack
- If $e \neq ')'$, append token to the right
- If $e == ')'$:
 - Find the last '('
 - Call `eval_simple_expr()` for this sub-expr
 - Remove the last '('
 - Append the sub-expr result to the right
- Post-process the last simple expression

```
def eval_expr(tokens):
    stack = []
    for e in tokens:
        if e == ')':
            reverse_idx = stack[::-1].index('(')
            last_open = len(stack) - reverse_idx - 1
            res = eval_simple_expr(stack[last_open+1:])
            stack = stack[:last_open]
            stack.append(str(res))
        else:
            stack.append(e)
    return eval_simple_expr(stack)

print(eval_expr(parse("2*(3.4-(1.2+0.2))/0.5")))
# 8.0
```

```
['2', '*', '(', '3.4', '-', '(', '1.2', '+', '0.2' ]
```


Evaluate Codes

- Given a list of assignments:
 - var_name = expression
- Store the var_name and result in a dictionary.
- Source list? Control variable?
- Accumulator?

```
variables = {}  
codes = [  
    'a=2*(3+2)',  
    'b=4+1']  
  
def eval_code(codes):  
    pass  
  
eval_code(codes)  
print(variables)  
# {'a ': 10.0, 'b ': 5.0}
```

Evaluate Codes

- Source list: codes
- Each line of code is split by "="
- Accumulator is a dictionary

```
variables = {}
codes = [
    'a=2*(3+2)',
    'b=4+1']

def eval_code(codes):
    for s in codes:
        var_name, expr = s.split("=")
        variables[var_name] = eval_expr(parse(expr))

eval_code(codes)
print(variables)
# {'a ': 10.0, 'b ': 5.0}
```

Evaluate Codes

- Replace var_name with its values
- Source list?
- Control variable?
- How to replace the value, eg:
 - tokens[1] = variables['a']?
 - tokens[6] = variables['b']?

```
variables = {}
codes = [
    'a=2*(3+2)',
    'b=4+1',
    'c=(a-5.0)/b']

def replace_var_name(tokens):
    pass

def eval_code(codes):
    for s in codes:
        var_name, expr = s.split("=")
        tokens = parse(expr)
        # ['(', 'a', '-', '5.0', ')', '/', 'b']
        replace_var_name(tokens)
        # ['(', '10.0', '-', '5.0', ')', '/', '5.0']
        variables[var_name] = eval_expr(tokens)

eval_code(codes)
print(variables)
# {'a': 10.0, 'b': 5.0, 'c': 1.0}
```

Evaluate Codes

- Use for loop to loop through each token (e)
- How to replace the value, eg:
 - `tokens[index_of(e)] = variables[e]`

```
variables = {}
codes = [
    'a=2*(3+2)',
    'b=4+1',
    'c=(a-5.0)/b']

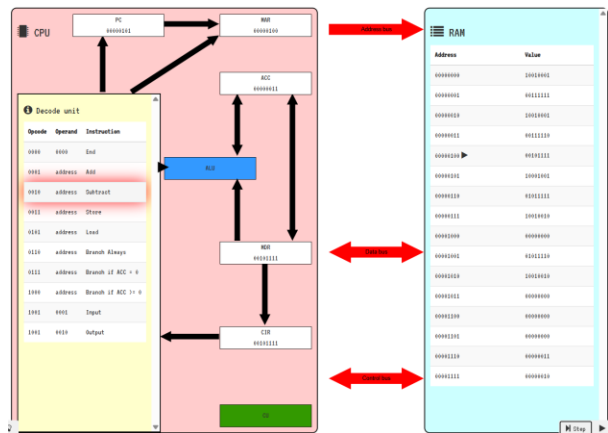
def replace_var_name(tokens):
    for e in tokens:
        if e in variables:
            tokens[tokens.index(e)] = str(variables[e])

def eval_code(codes):
    for s in codes:
        var_name, expr = s.split("=")
        tokens = parse(expr)
        # ['(', 'a', '-', '5.0', ')', '/', 'b']
        replace_var_name(tokens)
        # ['(', '10.0', '-', '5.0', ')', '/', '5.0']
        variables[var_name] = eval_expr(tokens)

eval_code(codes)
print(variables)
# {'a': 10.0, 'b': 5.0, 'c': 1.0}
```

In Lect. 1, Machine Code vs Python Code

- Different levels of abstraction
 - Machine code: CPU, register, RAM
 - Python: Parsing nums, ops, variables, then execute operations



```
0000: 10010001 Input eg. 2
0001: 00111111 Store [1111]
0010: 10010001 Input eg. 3
0011: 00111110 Store [1110]
0100: 00101111 Subtract [1111]
0101: 10001001 ACC>=0? [1001]
0110: 01011111 Load [1111]
0111: 10010010 Output
1000: 00000000 End
1001: 01011110 Load [1110]
1010: 10010010 Output 3
1011: 00000000 End
1100:
1101:
1110: 00000011
1111: 00000010
```

```
a = int(input("a"))
b = int(input("b"))
print(max(a,b))
```

Python Interpreter

Operating System (OS)

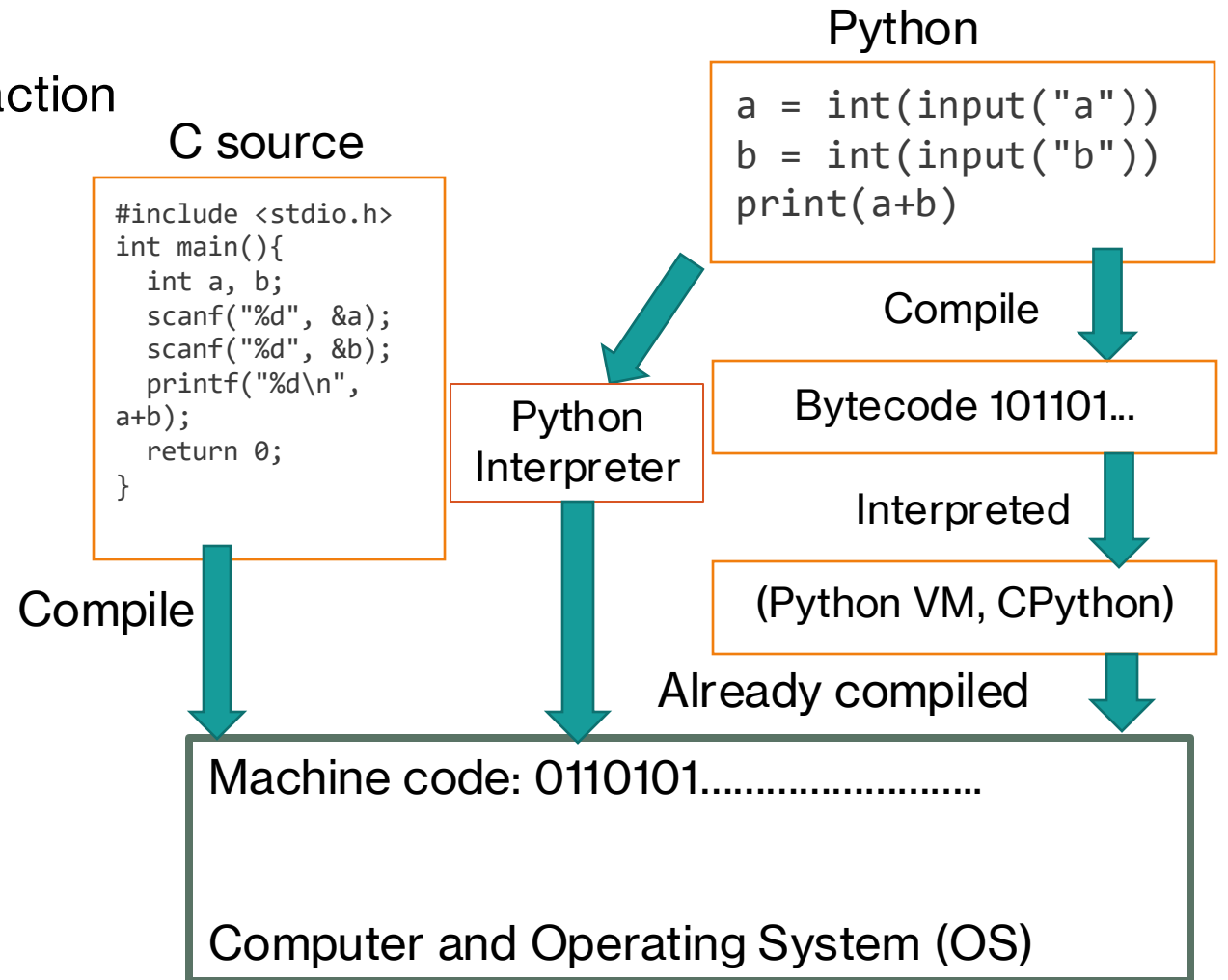
Computer Hardware

Image is obtained from the CPU simulator at the following published website:
<https://tools.withcode.uk/cpu/?ram=913f913e2f895f92005e920000000000>

From High-level Language to Machine code

- Write machine code is hard, due to its abstraction
- C source compiled to machine code
- Python source is **interpreted line by line**
- Python source compiled to bytecode
 - Compile translate the entire source to target

```
codes = [  
    'a=2*(3+2)',  
    'b=4+1',  
    'c=(a-5.0)/b']  
...  
def eval_code(codes):  
    for s in codes:  
        var_name, expr = s.split("=")  
        tokens = parse(expr)  
        # ['(', 'a', '-', '5.0', ')', '/', 'b']  
        replace_var_name(tokens)  
        # ['(', '10.0', '-', '5.0', ')', '/', '5.0']  
        variables[var_name] = eval_expr(tokens)
```



Python Source to Bytecode

- Use function.__code__ to obtain bytecode
 - Consts, variables, function name
- This example is based Python 3.10

```
00: 01100100 1
01: 01111101 0
02: 01111100 0
03: 01100100 2
04: 00011000 0
05: 01111100 0
06: 01100100 1
07: 00011011 0
08: 00011011 0
09: 01111101 1
10: 01110100 0
11: 01111100 1
12: 10000011 1
13: 00000001 0
14: 01100100 0
15: 01010011 0
```

```
def foo():
    a=2
    b=(a-5.0)/(a/2)
    print(b)
```

```
code = foo.__code__.co_code
consts = foo.__code__.co_consts
varnames = foo.__code__.co_varnames
names = foo.__code__.co_names
print(consts)      #(None, 2, 5.0)
print(varnames)    #('a', 'b')
print(names)       #('print',)
print(len(code))   #32
```

```
for i in range(16):
    opcode, operand = code[i*2], code[i*2+1]
    print(f"{i:02}: {opcode:08b} {operand}")
```

Interpreting Python Bytecode

- Load const #1 to top of stack

Stack from bottom -> top:
[2]

```
00: 01100100 LOAD_CONST 1
01: 01111101 STORE_VAR 0
02: 01111100 LOAD_VAR 0
03: 01100100 LOAD_CONST 2
04: 00011000 SUBTRACT 0
05: 01111100 LOAD_VAR 0
06: 01100100 LOAD_CONST 1
07: 00011011 DIVISION 0
08: 00011011 DIVISION 0
09: 01111101 STORE_VAR 1
10: 01110100 LOAD_GLOBAL 0
11: 01111100 LOAD_VAR 1
12: 10000011 CALL_FUNC 1
13: 00000001 POP 0
14: 01100100 LOAD_CONST 0
15: 01010011 RETURN 0
```

```
def foo():
    a=2
    b=(a-5.0)/(a/2)
    print(b)

...
print(consts)      #(None, 2, 5.0)
print(varnames)    #('a', 'b')
print(names)       #('print',)
```


Interpreting Python Bytecode

- Pop and store value to variable #0

Stack from bottom -> top:

Before: [2]

After: []

Variable:

a = 2

```
00: 01100100 LOAD_CONST 1
01: 01111101 STORE_VAR 0
02: 01111100 LOAD_VAR 0
03: 01100100 LOAD_CONST 2
04: 00011000 SUBTRACT 0
05: 01111100 LOAD_VAR 0
06: 01100100 LOAD_CONST 1
07: 00011011 DIVISION 0
08: 00011011 DIVISION 0
09: 01111101 STORE_VAR 1
10: 01110100 LOAD_GLOBAL 0
11: 01111100 LOAD_VAR 1
12: 10000011 CALL_FUNC 1
13: 00000001 POP 0
14: 01100100 LOAD_CONST 0
15: 01010011 RETURN 0
```

```
def foo():
    a=2
    b=(a-5.0)/(a/2)
    print(b)

...
print(consts)      #(None, 2, 5.0)
print(varnames)    #('a', 'b')
print(names)       #('print',)
```

Interpreting Python Bytecode

- Load variable #0 to stack

Stack from bottom -> top:

Before: []

After: [2]

Variable:

a = 2

```
00: 01100100 LOAD_CONST 1
01: 01111101 STORE_VAR 0
02: 01111100 LOAD_VAR 0
03: 01100100 LOAD_CONST 2
04: 00011000 SUBTRACT 0
05: 01111100 LOAD_VAR 0
06: 01100100 LOAD_CONST 1
07: 00011011 DIVISION 0
08: 00011011 DIVISION 0
09: 01111101 STORE_VAR 1
10: 01110100 LOAD_GLOBAL 0
11: 01111100 LOAD_VAR 1
12: 10000011 CALL_FUNC 1
13: 00000001 POP 0
14: 01100100 LOAD_CONST 0
15: 01010011 RETURN 0
```

```
def foo():
    a=2
    b=(a-5.0)/(a/2)
    print(b)

...
print(consts)      #(None, 2, 5.0)
print(varnames)    #('a', 'b')
print(names)       #('print',)
```

Interpreting Python Bytecode

- Load const #2 to stack

Stack from bottom -> top:

Before: [2]

After: [2, 5.0]

Variable:

a = 2

```
00: 01100100 LOAD_CONST 1
01: 01111101 STORE_VAR 0
02: 01111100 LOAD_VAR 0
03: 01100100 LOAD_CONST 2
04: 00011000 SUBTRACT 0
05: 01111100 LOAD_VAR 0
06: 01100100 LOAD_CONST 1
07: 00011011 DIVISION 0
08: 00011011 DIVISION 0
09: 01111101 STORE_VAR 1
10: 01110100 LOAD_GLOBAL 0
11: 01111100 LOAD_VAR 1
12: 10000011 CALL_FUNC 1
13: 00000001 POP 0
14: 01100100 LOAD_CONST 0
15: 01010011 RETURN 0
```

```
def foo():
    a=2
    b=(a-5.0)/(a/2)
    print(b)

...
print(consts)      #(None, 2, 5.0)
print(varnames)    #('a', 'b')
print(names)       #('print',)
```

Interpreting Python Bytecode

- Pop the top 2, subtract, push back

Stack from bottom -> top:

Before: [2, 5.0]

After: [-3.0]

Variable:

a = 2

```
00: 01100100 LOAD_CONST 1
01: 01111101 STORE_VAR 0
02: 01111100 LOAD_VAR 0
03: 01100100 LOAD_CONST 2
04: 00011000 SUBTRACT 0
05: 01111100 LOAD_VAR 0
06: 01100100 LOAD_CONST 1
07: 00011011 DIVISION 0
08: 00011011 DIVISION 0
09: 01111101 STORE_VAR 1
10: 01110100 LOAD_GLOBAL 0
11: 01111100 LOAD_VAR 1
12: 10000011 CALL_FUNC 1
13: 00000001 POP 0
14: 01100100 LOAD_CONST 0
15: 01010011 RETURN 0
```

```
def foo():
    a=2
    b=(a-5.0)/(a/2)
    print(b)
...
print(consts)      #(None, 2, 5.0)
print(varnames)    #('a', 'b')
print(names)       #('print',)
```

Interpreting Python Bytecode

- Load variable #0

Stack from bottom -> top:

Before: [-3.0]

After: [-3.0, 2]

Variable:

a = 2

```
00: 01100100 LOAD_CONST 1
01: 01111101 STORE_VAR 0
02: 01111100 LOAD_VAR 0
03: 01100100 LOAD_CONST 2
04: 00011000 SUBTRACT 0
05: 01111100 LOAD_VAR 0
06: 01100100 LOAD_CONST 1
07: 00011011 DIVISION 0
08: 00011011 DIVISION 0
09: 01111101 STORE_VAR 1
10: 01110100 LOAD_GLOBAL 0
11: 01111100 LOAD_VAR 1
12: 10000011 CALL_FUNC 1
13: 00000001 POP 0
14: 01100100 LOAD_CONST 0
15: 01010011 RETURN 0
```

```
def foo():
    a=2
    b=(a-5.0)/(a/2)
    print(b)

...
print(consts)      #(None, 2, 5.0)
print(varnames)    #('a', 'b')
print(names)       #('print',)
```

Interpreting Python Bytecode

- Load const #1

Stack from bottom -> top:

Before: [-3.0, 2]

After: [-3.0, 2, 2]

Variable:

a = 2

```
00: 01100100 LOAD_CONST 1
01: 01111101 STORE_VAR 0
02: 01111100 LOAD_VAR 0
03: 01100100 LOAD_CONST 2
04: 00011000 SUBTRACT 0
05: 01111100 LOAD_VAR 0
06: 01100100 LOAD_CONST 1
07: 00011011 DIVISION 0
08: 00011011 DIVISION 0
09: 01111101 STORE_VAR 1
10: 01110100 LOAD_GLOBAL 0
11: 01111100 LOAD_VAR 1
12: 10000011 CALL_FUNC 1
13: 00000001 POP 0
14: 01100100 LOAD_CONST 0
15: 01010011 RETURN 0
```

```
def foo():
    a=2
    b=(a-5.0)/(a/2)
    print(b)

...
print(consts)      #(None, 2, 5.0)
print(varnames)    #('a', 'b')
print(names)       #('print',)
```

Interpreting Python Bytecode

- Pop the top 2, division, push back

Stack from bottom -> top:

Before: [-3.0, 2, 2]

After: [-3.0, 1]

Variable:

a = 2

```
00: 01100100 LOAD_CONST 1
01: 01111101 STORE_VAR 0
02: 01111100 LOAD_VAR 0
03: 01100100 LOAD_CONST 2
04: 00011000 SUBTRACT 0
05: 01111100 LOAD_VAR 0
06: 01100100 LOAD_CONST 1
07: 00011011 DIVISION 0
08: 00011011 DIVISION 0
09: 01111101 STORE_VAR 1
10: 01110100 LOAD_GLOBAL 0
11: 01111100 LOAD_VAR 1
12: 10000011 CALL_FUNC 1
13: 00000001 POP 0
14: 01100100 LOAD_CONST 0
15: 01010011 RETURN 0
```

```
def foo():
    a=2
    b=(a-5.0)/(a/2)
    print(b)

...
print(consts)      #(None, 2, 5.0)
print(varnames)    #('a', 'b')
print(names)       #('print',)
```

Interpreting Python Bytecode

- Pop the top 2, division, push back

Stack from bottom -> top:

Before: [-3.0, 1]

After: [-3.0]

Variable:

a = 2

```
00: 01100100 LOAD_CONST 1
01: 01111101 STORE_VAR 0
02: 01111100 LOAD_VAR 0
03: 01100100 LOAD_CONST 2
04: 00011000 SUBTRACT 0
05: 01111100 LOAD_VAR 0
06: 01100100 LOAD_CONST 1
07: 00011011 DIVISION 0
08: 00011011 DIVISION 0
09: 01111101 STORE_VAR 1
10: 01110100 LOAD_GLOBAL 0
11: 01111100 LOAD_VAR 1
12: 10000011 CALL_FUNC 1
13: 00000001 POP 0
14: 01100100 LOAD_CONST 0
15: 01010011 RETURN 0
```

```
def foo():
    a=2
    b=(a-5.0)/(a/2)
    print(b)

...
print(consts)      #(None, 2, 5.0)
print(varnames)    #('a', 'b')
print(names)       #('print',)
```


Interpreting Python Bytecode

- Pop and store to variable #1

Stack from bottom -> top:

Before: [-3.0]

After: []

Variable:

a = 2

b = -3.0

```
00: 01100100 LOAD_CONST 1
01: 01111101 STORE_VAR 0
02: 01111100 LOAD_VAR 0
03: 01100100 LOAD_CONST 2
04: 00011000 SUBTRACT 0
05: 01111100 LOAD_VAR 0
06: 01100100 LOAD_CONST 1
07: 00011011 DIVISION 0
08: 00011011 DIVISION 0
09: 01111101 STORE_VAR 1
10: 01110100 LOAD_GLOBAL 0
11: 01111100 LOAD_VAR 1
12: 10000011 CALL_FUNC 1
13: 00000001 POP 0
14: 01100100 LOAD_CONST 0
15: 01010011 RETURN 0
```

```
def foo():
    a=2
    b=(a-5.0)/(a/2)
    print(b)

...
print(consts)      #(None, 2, 5.0)
print(varnames)    #('a', 'b')
print(names)       #('print',)
```

Interpreting Python Bytecode

- Load the names #0

Stack from bottom -> top:

Before: []

After: ['print']

Variable:

a = 2

b = -3.0

```
00: 01100100 LOAD_CONST 1
01: 01111101 STORE_VAR 0
02: 01111100 LOAD_VAR 0
03: 01100100 LOAD_CONST 2
04: 00011000 SUBTRACT 0
05: 01111100 LOAD_VAR 0
06: 01100100 LOAD_CONST 1
07: 00011011 DIVISION 0
08: 00011011 DIVISION 0
09: 01111101 STORE_VAR 1
10: 01110100 LOAD_GLOBAL 0
11: 01111100 LOAD_VAR 1
12: 10000011 CALL_FUNC 1
13: 00000001 POP 0
14: 01100100 LOAD_CONST 0
15: 01010011 RETURN 0
```

```
def foo():
    a=2
    b=(a-5.0)/(a/2)
    print(b)

...
print(consts)      #(None, 2, 5.0)
print(varnames)    #('a', 'b')
print(names)       #('print',)
```

Interpreting Python Bytecode

- Load the variable #1

Stack from bottom -> top:

Before: ['print']

After: ['print', -3.0]

Variable:

a = 2

b = -3.0

```
00: 01100100 LOAD_CONST 1
01: 01111101 STORE_VAR 0
02: 01111100 LOAD_VAR 0
03: 01100100 LOAD_CONST 2
04: 00011000 SUBTRACT 0
05: 01111100 LOAD_VAR 0
06: 01100100 LOAD_CONST 1
07: 00011011 DIVISION 0
08: 00011011 DIVISION 0
09: 01111101 STORE_VAR 1
10: 01110100 LOAD_GLOBAL 0
11: 01111100 LOAD_VAR 1
12: 10000011 CALL_FUNC 1
13: 00000001 POP 0
14: 01100100 LOAD_CONST 0
15: 01010011 RETURN 0
```

```
def foo():
    a=2
    b=(a-5.0)/(a/2)
    print(b)

...
print(consts)      #(None, 2, 5.0)
print(varnames)    #('a', 'b')
print(names)       #('print',)
```

Interpreting Python Bytecode

- Pop top 2, call func, push back result

Stack from bottom -> top:

Before: ['print', -3.0]

After: [None]

Variable:

a = 2

b = -3.0

Display:

-3.0

```
00: 01100100 LOAD_CONST 1
01: 01111101 STORE_VAR 0
02: 01111100 LOAD_VAR 0
03: 01100100 LOAD_CONST 2
04: 00011000 SUBTRACT 0
05: 01111100 LOAD_VAR 0
06: 01100100 LOAD_CONST 1
07: 00011011 DIVISION 0
08: 00011011 DIVISION 0
09: 01111101 STORE_VAR 1
10: 01110100 LOAD_GLOBAL 0
11: 01111100 LOAD_VAR 1
12: 10000011 CALL_FUNC 1
13: 00000001 POP 0
14: 01100100 LOAD_CONST 0
15: 01010011 RETURN 0
```

```
def foo():
    a=2
    b=(a-5.0)/(a/2)
    print(b)

...
print(consts)      #(None, 2, 5.0)
print(varnames)    #('a', 'b')
print(names)       #('print',)
```

Interpreting Python Bytecode

- Pop the top value

Stack from bottom -> top:

Before:[None]

After: []

Variable:

a = 2

b = -3.0

Display:

-3.0

```
00: 01100100 LOAD_CONST 1
01: 01111101 STORE_VAR 0
02: 01111100 LOAD_VAR 0
03: 01100100 LOAD_CONST 2
04: 00011000 SUBTRACT 0
05: 01111100 LOAD_VAR 0
06: 01100100 LOAD_CONST 1
07: 00011011 DIVISION 0
08: 00011011 DIVISION 0
09: 01111101 STORE_VAR 1
10: 01110100 LOAD_GLOBAL 0
11: 01111100 LOAD_VAR 1
12: 10000011 CALL_FUNC 1
13: 00000001 POP 0
14: 01100100 LOAD_CONST 0
15: 01010011 RETURN 0
```

```
def foo():
    a=2
    b=(a-5.0)/(a/2)
    print(b)

...
print(consts)      #(None, 2, 5.0)
print(varnames)    #('a', 'b')
print(names)       #('print',)
```

Interpreting Python Bytecode

- Load const #0

Stack from bottom -> top:

Before: []

After: [None]

Variable:

a = 2

b = -3.0

Display:

-3.0

```
00: 01100100 LOAD_CONST 1
01: 01111101 STORE_VAR 0
02: 01111100 LOAD_VAR 0
03: 01100100 LOAD_CONST 2
04: 00011000 SUBTRACT 0
05: 01111100 LOAD_VAR 0
06: 01100100 LOAD_CONST 1
07: 00011011 DIVISION 0
08: 00011011 DIVISION 0
09: 01111101 STORE_VAR 1
10: 01110100 LOAD_GLOBAL 0
11: 01111100 LOAD_VAR 1
12: 10000011 CALL_FUNC 1
13: 00000001 POP 0
14: 01100100 LOAD_CONST 0
15: 01010011 RETURN 0
```

```
def foo():
    a=2
    b=(a-5.0)/(a/2)
    print(b)

...
print(consts)      #(None, 2, 5.0)
print(varnames)    #('a', 'b')
print(names)       #('print',)
```

Interpreting Python Bytecode

- Pop and return

Stack from bottom -> top:

Before: [None]

After: []

Variable:

a = 2

b = -3.0

Display:

-3.0

Return value:

None

```
00: 01100100 LOAD_CONST 1
01: 01111101 STORE_VAR 0
02: 01111100 LOAD_VAR 0
03: 01100100 LOAD_CONST 2
04: 00011000 SUBTRACT 0
05: 01111100 LOAD_VAR 0
06: 01100100 LOAD_CONST 1
07: 00011011 DIVISION 0
08: 00011011 DIVISION 0
09: 01111101 STORE_VAR 1
10: 01110100 LOAD_GLOBAL 0
11: 01111100 LOAD_VAR 1
12: 10000011 CALL_FUNC 1
13: 00000001 POP 0
14: 01100100 LOAD_CONST 0
15: 01010011 RETURN 0
```

```
def foo():
    a=2
    b=(a-5.0)/(a/2)
    print(b)

...
print(consts)      #(None, 2, 5.0)
print(varnames)    #('a', 'b')
print(names)       #('print',)
```

Summary

- Parsing code
 - Reading Grammar
 - Interpreting code
 - Python VM

